

MODULE 5.1 · Concept 2 of 7

LLM-Based Planning

How language models generate plans through prompting — and where they fail

10 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

TH-000005

Prerequisite

AI-AG-IN-TH-000006

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — HOW

LLM Plan Generation

Given a goal, produce a step-by-step natural language plan

How LLMs Generate Plans

Goal description in → step-by-step plan out — via prompting alone

Input (user goal):

"Plan a 3-day team offsite in Goa for 10 people with a budget of ₹5 lakhs"

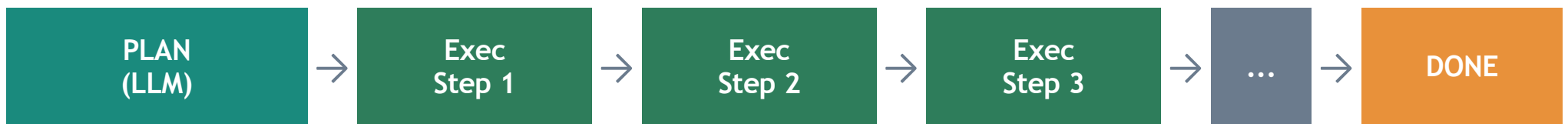
Output (LLM-generated plan):

Step 1: Search for Goa resorts with group rates for 10 people, 3 nights
Step 2: Compare options and select one within ₹2.5L accommodation budget
Step 3: Book flights BLR→GOA for 10 people (budget: ₹1.5L)
Step 4: Plan activities: Day 1 arrival, Day 2 team building, Day 3 leisure
Step 5: Book restaurants for group dinners (budget: ₹50K)
Step 6: Create itinerary document and share with team

No formal logic needed. The LLM understands the goal in natural language and produces executable steps.

Plan-Then-Execute: Generate Full Plan First

Create the complete plan, then execute each step sequentially



Advantages:

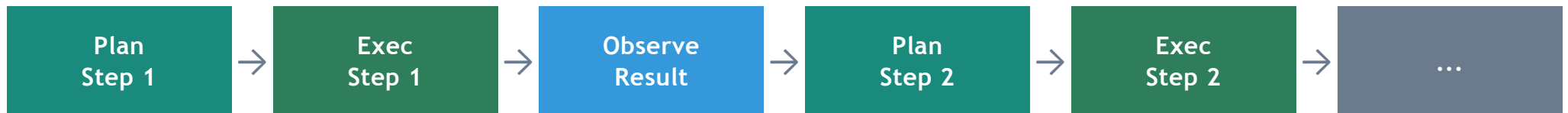
- Agent knows the full path upfront
- Can estimate total cost/time before starting
- User can review and approve plan
- Easier to parallelize independent steps

Disadvantages:

- Plan may be wrong — all steps wasted
- Cannot adapt to mid-execution surprises
- Longer time-to-first-action
- Entire plan regenerated if step 1 fails

Interleaved Planning: Plan One Step Ahead

Plan → Execute → Observe → Replan at each iteration (ReAct + planning)



Advantages:

- Adapts to real-time results
- Each step informed by actual data
- Recovers from failures gracefully
- No wasted planning on wrong assumptions

Disadvantages:

- No global view of the full path
- May wander or lose direction (goal drift)
- Cannot estimate total cost upfront
- Harder to parallelize steps

Plan-Then-Execute vs. Interleaved: When to Use Which

A practical decision table

Factor	Plan-Then-Execute	Interleaved Planning
User approval needed	✓ User reviews full plan	✗ User sees one step at a time
Environment is stable	✓ Plan stays valid	Overkill — plan once is enough
Environment is dynamic	✗ Plan goes stale quickly	✓ Adapts to changes
Cost estimation needed	✓ Can estimate before starting	✗ Cannot predict total cost
Complex multi-step	✓ Better coordination	May lose the big picture
Failure likely	✗ Entire plan scrapped on failure	✓ Replans from current state
Simple task	Overkill	✓ Just ReAct — no plan needed

Chain-of-Thought Planning

'Think step by step' as a planning strategy — the simplest LLM planning technique

Planning Prompt:

"You are a planning agent. Given the user's goal, think step by step about what needs to happen. List each step as a numbered action. For each step, specify:

- What tool to use
- What information you need
- What the expected output is
- Which previous steps this depends on"

This is the simplest LLM planning technique — no special framework needed. Just a well-structured prompt.

Enhancement — Plan decomposition prompt (structured output):

"Output the plan as JSON: [{step_num, description, tool, depends_on, expected_output}]"

Worked Example: 'Plan a Team Offsite in Goa'

Full plan generated, with dependencies and execution order

#	Description	Tool	Depends On	Expected Output
1	Search Goa resorts with group rates	web_search	—	List of 5 resorts with prices
2	Compare and select best resort	LLM reasoning	Step 1	Selected resort + justification
3	Search group flights BLR→GOA	web_search	—	Flight options with prices
4	Book resort + flights	booking_api	Steps 2,3	Confirmation numbers
5	Plan daily activity schedule	LLM reasoning	Step 2	3-day itinerary
6	Create and share itinerary doc	doc_writer	Steps 4,5	PDF shared with team

Steps 1 and 3 are PARALLEL (no dependency). Steps 4 and 5 can also run in parallel. Step 6 waits for both.

SECTION 2 — LIMITS

When LLM Plans Fail

Plausible but infeasible — the core limitation of LLM planning

Plan Verification: Catch Errors Before Execution

Ask the LLM to critique its own plan — a lightweight safety net

Verification Prompt (sent AFTER plan generation):

"Review this plan for the following issues:

1. Are any steps out of order? (Step N needs data from Step M that hasn't run yet)
2. Are there missing steps? (Something needed but not included)
3. Are there impossible steps? (Requires a tool/capability that doesn't exist)
4. Does the total estimated cost exceed the budget?
5. Are there redundant steps that can be eliminated?

For each issue found, suggest a specific fix."

This costs one extra LLM call (~\$0.01) but catches ordering errors and missing steps before wasting execution budget.

Limitation: Plausible But Infeasible Plans

LLMs generate plans that SOUND good but CAN'T execute

Example of a plausible but infeasible plan:

Goal: "Book the cheapest flight BLR→GOA for tomorrow"

LLM Plan:

- Step 1: Search flights BLR→GOA for tomorrow
- Step 2: Sort results by price
- Step 3: Book the cheapest option

Problem: Step 3 assumes the cheapest flight has available seats.

Reality: cheapest flight is sold out. Plan has no fallback.

Also: "sort results" is not a tool the agent has — it's LLM reasoning pretending to be an action.

LLMs don't check real-world constraints. They generate plans from training patterns, not from live system state.

More Limitations: Ordering Errors + No Constraint Propagation

Two more ways LLM plans go wrong

Ordering Errors

LLM generates:

- Step 1: Book the hotel
- Step 2: Search for hotels

Step 1 needs Step 2's output!

LLMs don't track data flow between steps. They pattern-match "booking tasks" without checking dependencies.

No Constraint Propagation

Budget: ₹15,000 total

- Step 1: Book flight (₹8,000)
 - Step 2: Book hotel (₹10,000)
 - Step 3: Book activities (₹5,000)
- Total: ₹23,000 > budget!

LLM doesn't carry running totals.
Each step is planned independently.
No resource tracking across steps.

Both failures are detectable with verification prompts or classical constraint checkers → Hybrid Planning (next concept).

LangChain PlanAndExecute Architecture

The production pattern for plan-then-execute agents

LangChain provides a built-in PlanAndExecute agent pattern:

```
from langchain.agents import PlanAndExecute, load_agent_executor
from langchain.agents.plan_and_execute import PlannerChain

# Planner: generates the plan (uses GPT-4o for quality)
planner = PlannerChain.from_llm(ChatOpenAI(model="gpt-4o"))

# Executor: runs each step (uses GPT-4o-mini for cost)
executor = load_agent_executor(ChatOpenAI(model="gpt-4o-mini"), tools)

# Combined: plan then execute
agent = PlanAndExecute(planner=planner, executor=executor)
result = agent.run("Plan a team offsite in Goa for 10 people, ₹5L budget")
```

Notice: planner uses GPT-4o (\$0.03) for quality; executor uses mini (\$0.001) for cost. Tiered routing in action.

Key Takeaways

The essential points from this concept

- LLMs generate plans from natural language goals — no formal logic needed.
- Two strategies: plan-then-execute (full plan first) vs. interleaved (one step at a time).
- CoT planning is the simplest technique — 'think step by step' with structured output.
- LLM plans can be plausible but infeasible: ordering errors, budget violations, missing constraints.
- Verification prompts catch many errors — one extra LLM call before execution saves wasted work.

Next: AI-AG-IN-TH-000007 — Hybrid Planning Approaches

AI-AG-IN-TH-000006 Complete

LLM-Based Planning

Concept 2 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi